

Breaking the Classics: A Cryptography Escape Room

Omar Ait Said | Intro to Cryptography – Prof. Imane Fouad | UM6P | October 2026

In this lab I went through a chain of classical ciphers (Caesar, substitution, Vigenère). Each one I broke gave me the clue for the next one. For each stage I wrote a small Python program. All the code is in the folder **Tp-Crypto** (warmup.py, substitution.py, frequency.py, room1_break.py, room2_vigenere.py).

Warm-Up – Before You Enter the Vault

Caesar shifts every letter by K positions: $C = (P + K) \bmod 26$ and $P = (C - K) \bmod 26$. I wrote one encrypt function and used it with $-K$ to decrypt.

1. Encrypt

With $K = 4$: CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ → **GVCTXS MW JYR YRXMP XLI TVSJIWWSV WECW UYMD**

2. Decrypt

With $K = 8$: GWCL JMBBMZ VWB JM CAQVO BWWTA BW AWTDM BPQA → **YOU'D BETTER NOT BE USING TOOLS TO SOLVE THIS**

3. The key is gone

Caesar only has 26 possible keys, so I don't need to find the key, I just try all of them and look for the line that makes sense. The program prints the 26 candidates and the right one is **K = 11: THE KEY IS UNKNOWN TRY BRUTE FORCE**. Brute force works here because the key space is tiny.

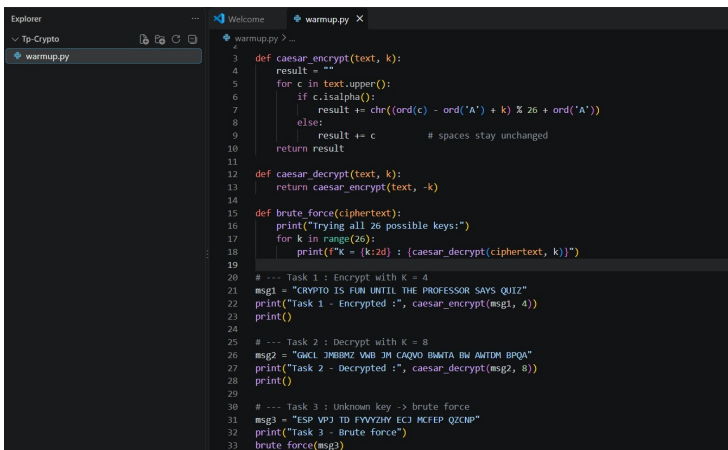


Figure 1 – warmup.py: Caesar encrypt/decrypt and brute force

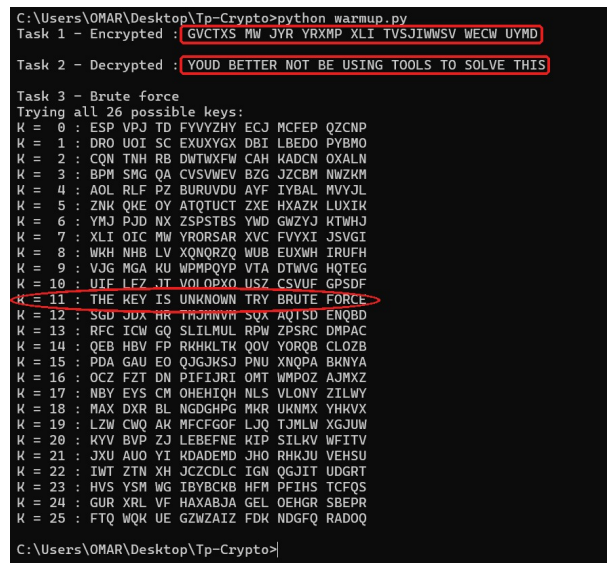


Figure 2 – Output: tasks 1 and 2, and all 26 keys (K = 11 circled)

Room 1 – The Letter Wall

Now each letter is replaced by another letter using a fixed alphabet (the same plain letter always gives the same cipher letter).

1. Learn the mechanism

KEEP THE SECRET SAFE → **LTTN EKT YTJRTE YQGT**

2. Decrypt

ERVYE DXVR SXFPJ → **TRUST YOUR LOGIC**

```

1 PLAIN = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
2 CIPHER = "QWJZTGFKPMLSBOXNCRVEYHIADU"
3
4 enc_table = dict(zip(PLAIN, CIPHER))
5 dec_table = dict(zip(CIPHER, PLAIN))
6
7 def sub_encrypt(text):
8     return "".join(enc_table.get(c, c) for c in text.upper())
9
10 def sub_decrypt(text):
11     return "".join(dec_table.get(c, c) for c in text.upper())
12
13 print("Task 1 - Encrypt 'KEEP THE SECRET SAFE' :", sub_encrypt("KEEP THE SECRET SAFE"))
14 print("Task 2 - Decrypt 'ERVYE DXVR SXFPJ' :", sub_decrypt("ERVYE DXVR SXFPJ"))

```

Figure 3 – substitution.py: encryption and decryption tables

```

C:\Users\OMAR\Desktop\Tp-Crypto>python substitution.py
Task 1 - Encrypt 'KEEP THE SECRET SAFE' : LTTN EKT YJT RTE YQGT
Task 2 - Decrypt 'ERVYE DXVR SXFPJ' : TRUST YOUR LOGIC

```

Figure 4 – Output of tasks 1 and 2

3. Now the substitution alphabet disappears

Answer: the key space is $26! \approx 4 \times 10^{26}$, so brute force is impossible. But substitution keeps the letter frequencies of English, it only changes the symbols. So the attacker counts the letters in the ciphertext: the most frequent one is probably E, the next one T, etc. This gives a partial key. Then he fixes the rest using short words (THE, AND, IS), repeated patterns and context.

I wrote **frequency.py**: it ignores spaces and punctuation, counts each letter, computes the percentage and sorts from most to least frequent. It returns the result so I can reuse it in Room 2.

```

1 # ===== FREQUENCY ANALYSIS TOOL =====
2 from collections import Counter
3
4 def frequency_analysis(text, show=True):
5     letters = [c for c in text.upper() if c.isalpha()] # ignore spaces & punctuation
6     total = len(letters)
7     counts = Counter(letters)
8     result = []
9     for letter, n in counts.most_common(): # most -> least frequent
10         pct = 100 * n / total
11         result.append((letter, n, pct))
12         if show:
13             print(f"letter : {letter} : {n:3d} ({pct:5.2f} %)")
14     if show:
15         print("Total letters:", total)
16     return result
17
18 if __name__ == "__main__":
19     test = input("Paste a ciphertext: ")
20     frequency_analysis(test)

```

Figure 5 – frequency.py: the frequency analysis tool

4. Break the intercepted message

I ran my tool on the ciphertext. The most frequent letters are T (17.3%), E, R, Y/X, Q, K, O, Z, P, F, S. I matched them with the order from the hint (E, T, R, S/O, A, H, N, D, I, G, L), which gave me a first mapping: T→E, E→T, R→R, Y→S, X→O, Q→A, K→H, O→N, Z→D, P→I, F→G, S→L.

With this mapping the text was already half readable. Then I guessed the missing letters from the words: SE_RET → SECRET (J→C), _ESSAGE → MESSAGE (B→M), _ARAGRA_H → PARAGRAPH (N→P), _RE_UEN_Y → FREQUENCY (G→F, C→Q, V→U, D→Y), RE_EAL → REVEAL (H→V), _HOLE → WHOLE (I→W), _EY → KEY (L→K), _UT → BUT (M→B).

Figure 6 – room1_break.py (part 1): frequencies + first mapping

Figure 7 – room1_break.py (part 2): completing the key from words

Figure 8 – Frequency table and partial decryption

Figure 9 – Full decryption and recovered key (clue circled)

Room 2 – Break the Vigenère Cipher

1. Recover the key

Since the key has 4 letters, letters at positions 0, 4, 8... use K1, positions 1, 5, 9... use K2, etc. So I split the ciphertext into 4 groups with `ct[i::4]`. Each group is just a Caesar cipher. I ran my frequency tool on each group, assumed the most frequent letter is E, and computed $\text{shift} = (\text{letter} - \text{E}) \bmod 26$.

Group	Most frequent	Shift	Key letter
1 (pos 0, 4, 8...)	G (11)	2	C
2 (pos 1, 5, 9...)	S (17)	14	O
3 (pos 2, 6, 10...)	H (9)	3	D
4 (pos 3, 7, 11...)	I (14)	4	E

Recovered key: CODE. Example with the table: row C, column E gives G, so $E + C = G$, which is exactly what I saw in group 1.

```

Explorer
  Tp-Crypto
    __pycache__
    frequency.py
    room1_break.py
    room2_vigenere.py
    substitution.py
    warmup.py

room2_vigenere.py > ...
1 from frequency import frequency_analysis
2
3 ct = ("VHhGQUVWHTRSUPGTHXJSVIPCHTUPHGIVVHPGHMITSOTRSOVUSYITMZL"
4       "GFHIXSUCVSDQSPFGVLQIOHEVHGMSDGJQRPWAQNGSNXJSPSUHIVGEXIPHVC"
5       "OPRPCBGVSLXVCUIECYITHKINSBAJSQXJSPUGDKPGHQAHEZHETSQXGFWL"
6       "GOFGGVAQFGIEZLTUS")
7
8 KEY_LENGTH = 4 # clue from Room 1
9
10 key = ""
11 for i in range(KEY_LENGTH):
12     group = ct[i::KEY_LENGTH] # split into 4 groups
13     print(f"\n== Group {i+1} (positions {i}, {i+KEY_LENGTH}, ...) ==")
14     freqs = frequency_analysis(group, show=False)
15     for letter, n, pct in freqs[5]:
16         print(f"{letter} : {n:3d} ({pct:5.2f} %)")
17     top = freqs[0][0] # most frequent cipher letter
18     shift = (ord(top) - ord('E')) % 26 # assume it is plaintext 'E'
19     key_letter = chr(shift + ord('A'))
20     print(f"Most frequent: {top} -> assume E -> shift {shift} -> key letter {key_letter}")
21     key += key_letter
22
23 print("\nRECOVERED KEY:", key)
24
25 def vigenere_decrypt(text, key):
26     out = ""
27     for i, c in enumerate(text):
28         k = ord(key[i % len(key)]) - ord('A')
29         out += chr((ord(c) - ord('A') - k) % 26 + ord('A'))
30     return out
31
32 plain = vigenere_decrypt(ct, key)
33 print("\nPLAINTEXT:\n", plain)

```

Figure 10 – room2_vigenere.py: splitting into 4 groups and computing each shift

```

C:\Users\OMAR\Desktop\Tp-Crypto\python room2_vigenere.py

== Group 1 (positions 0, 4, ...) ==
G : 11 (22.00 %)
U : 6 (12.00 %)
V : 5 (10.00 %)
J : 5 (10.00 %)
T : 4 (8.00 %)
Most frequent: G -> assume E -> shift 2 -> key letter U

== Group 2 (positions 1, 5, ...) ==
M : 4 (8.00 %)
F : 4 (8.00 %)
V : 3 (6.00 %)
C : 3 (6.00 %)
Most frequent: 5 -> assume E -> shift 14 -> key letter U

== Group 3 (positions 2, 6, ...) ==
H : 5 (10.00 %)
V : 5 (10.00 %)
O : 5 (10.00 %)
U : 4 (8.00 %)
W : 4 (8.00 %)
Most frequent: H -> assume E -> shift 3 -> key letter D

== Group 4 (positions 3, 7, ...) ==
I : 10 (20.00 %)
X : 5 (10.00 %)
G : 5 (10.00 %)
P : 4 (8.00 %)
W : 3 (6.00 %)
Most frequent: I -> assume E -> shift 4 -> key letter E

RECOVERED KEY: CODE
PLAINTEXT:
THESECREETKEEPLTHESENOTESBECAUSETHELETTEREAPPEARSEVERYWHEREEVERYTEAMMEMBERSHOULDCHECKEACHCOLUMNSEEKTHEMOSTFREQUENTSMBOLANDUSETITTORECOVERTHE
ESSAGEBECOMESCLEARENTERTHEACCESSWORDECLIPSE

```

Figure 11 – Output: key letter of each group, key CODE and access word

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Figure 12 – Vigenère table: row C, column E → G

2. Decrypt the message

With the key CODE I decrypted everything ($P = (C - K) \bmod 26$). With spaces added: THE SECRET KEEPER LEFT THESE NOTES BECAUSE THE LETTER E APPEARS EVERYWHERE. EVERY TEAM MEMBER SHOULD CHECK EACH COLUMN, SEEK THE MOST FREQUENT SYMBOL AND USE IT TO RECOVER THE KEY. WHEN THE MESSAGE BECOMES CLEAR, ENTER THE ACCESS WORD ECLIPSE.

3. Find the hidden access word

Access word (final flag): **ECLIPSE**

Conclusion – Why classical ciphers are insecure

- **Caesar**: only 26 keys, so brute force breaks it instantly.
- **Substitution**: huge key space ($26!$), but it keeps the letter frequencies, so frequency analysis breaks it.
- **Vigenère**: hides frequencies better, but once you know the key length it becomes a few Caesar ciphers, and each one falls to frequency analysis.

The lesson: a big key space is not enough. A secure cipher must also hide all the statistics of the plaintext, which is what modern ciphers like AES do.